

**UNIVERSIDAD NACIONAL DE SAN ANTONIO ABAD DEL CUSCO**  
**FACULTAD DE INGENIERÍA ELÉCTRICA, ELECTRÓNICA, INFORMÁTICA Y MECÁNICA**  
**ESCUELA PROFESIONAL DE INGENIERÍA INFORMÁTICA Y DE SISTEMAS**



**DOCUMENTO DE ESPECIFICACIÓN Y DISEÑO ARQUITECTÓNICO (V2)**

*Resolución de Ejercicios Ágiles, Requerimientos IEEE 830, Modelado UML y Base de Datos Relacional*

---

**“SISTEMA INTEGRAL DE GESTIÓN DE CAPACITACIONES (SIGC-CUSCO)”**

**Incremento Iterativo – Versión V2.0**

---

**Asignatura:** INGENIERÍA DE SOFTWARE I

**Docente:** Ing. Lisha Sabah Diaz Caceres

**Enfoque:** Desarrollo Incremental e Iterativo – Ingeniería de Requisitos

**Presentado por:**

- |                                    |        |
|------------------------------------|--------|
| • Choquenaira Quispe, Noe Franklin | 133962 |
| • Yaranga Achahui, Aldo            | 103179 |
| • Ccama Enriquez, Carolay          | 210921 |

## Índice

|                                                                                           |          |
|-------------------------------------------------------------------------------------------|----------|
| <b>1. Resolución de Ejercicios Prácticos de Gestión de Flujo Ágil</b>                     | <b>2</b> |
| 1.1. Ejercicio 1: Una Revisión Saturada (Gestión de Flujo y Cuellos de Botella) . . . . . | 2        |
| 1.2. Ejercicio 2: Coordinar una Entrega (Objetivo de Ciclo y Roles) . . . . .             | 3        |
| <b>2. Especificación Formal de Requerimientos de Software</b>                             | <b>3</b> |
| 2.1. Requerimientos Funcionales (RF) . . . . .                                            | 4        |
| 2.2. Requerimientos No Funcionales (RNF) según ISO/IEC 25010 . . . . .                    | 5        |
| <b>3. Modelado Visual del Sistema (UML)</b>                                               | <b>5</b> |
| 3.1. Diagrama de Casos de Uso del Sistema (UML) . . . . .                                 | 5        |
| 3.2. Diagrama de Clases del Dominio y Multiplicidades . . . . .                           | 6        |
| <b>4. Diseño de Base de Datos y Esquema Relacional</b>                                    | <b>6</b> |
| 4.1. Explicación Didáctica de las Conexiones Relacionales . . . . .                       | 6        |
| 4.2. Esquema Físico de Base de Datos (Script DDL en PostgreSQL) . . . . .                 | 7        |
| <b>5. Herramientas para Diagramado Rápido y Colaborativo</b>                              | <b>9</b> |
| <b>6. Plan de Evolución Incremental e Iterativo por Sprints</b>                           | <b>9</b> |

# 1. Resolución de Ejercicios Prácticos de Gestión de Flujo Ágil

A continuación, se presentan las respuestas fundamentadas a los dos ejercicios prácticos desarrollados en clase, aplicadas de manera directa y pedagógica a la realidad de nuestro proyecto **SIGC-CUSCO**.

## 1.1. Ejercicio 1: Una Revisión Saturada (Gestión de Flujo y Cuellos de Botella)

**Escenario Planteado:** *El límite de la columna Revisión (Review) es 1 y ya existe un ítem esperando en cola.*

### 1. ¿Conviene iniciar la tarea de recordatorios? Justificación:

- **Respuesta: NO conviene iniciarla.**
- **Fundamento Ágil:** En el marco Kanban rige el principio fundamental: “*Stop starting, start finishing*” (Dejar de empezar cosas nuevas y enfocarse en terminar lo que está en curso). Si la columna de *Revisión* tiene un límite WIP (*Work In Progress*) de 1 y ya se encuentra saturada, abrir una nueva tarjeta de desarrollo (como “Módulo de Recordatorios”) generará acumulación de inventario intermedio no validado, aumentará el tiempo de ciclo (*Lead Time*) del equipo y saturará aún más el cuello de botella.

### 2. Acuerdo de Colaboración: ¿Cómo ayudaremos a terminar la revisión?

- Se aplica la técnica de **Swarming / Mob Review**: El desarrollador que terminó su tarjeta se pone a disposición del revisor en lugar de iniciar código nuevo.
- El compañero apoya realizando pruebas cruzadas (*peer testing*) en local, verificando que los criterios de aceptación se cumplan y ayudando a resolver inmediatamente las observaciones de código detectadas. De esta manera, el ítem en revisión se promueve rápidamente a **Done** y el flujo se reactiva de forma saludable.

### 3. Protocolo de Actuación ante una Tarjeta Bloqueada:

- **Señalización Visual Inmediata:** Se le asigna una etiqueta roja destacada con el estado [BLOQUEADA] en el tablero.
- **Registro de Causa Raíz:** Se documenta en la tarjeta la causa exacta (ej. “*Falta acceso a la API del validador de DNI*” o “*Falta confirmación de firmas de la municipalidad*”).
- **Regla de Escalamiento:** Una tarjeta bloqueada no cuenta para saturar el desarrollo de quien la ejecutaba, pero adquiere máxima prioridad para el facilitador del equipo, quien debe gestionar el impedimento en un plazo no mayor a 24 horas.

#### Políticas de WIP y Bloqueo Adoptadas en SIGC-CUSCO

- **Límites WIP Oficiales:** *To Do* = 5 máx. | *In Progress* = 2 máx. | *Review / QA* = 1 estricto | *Done* = Ilimitado.
- **Política de Bloqueo:** Si un ítem se bloquea, el equipo dedica la primera mitad de la reunión de coordinación (*Daily*) a destrabarlo antes de iniciar cualquier otra discusión.

## 1.2. Ejercicio 2: Coordinar una Entrega (Objetivo de Ciclo y Roles)

**Escenario Planteado:** *El equipo debe demostrar una funcionalidad utilizable en una entrega incremental.*

### 1. Objetivo de Ciclo (Orientado a Valor, no lista de tareas):

*“Permitir que un participante de Cusco complete exitosamente su inscripción en línea a una capacitación municipal (ej. curso de SIGA) mediante la validación de su DNI en tiempo real, obteniendo en pantalla su credencial digital con código QR funcional para el control de asistencia.”*

*Justificación:* No es una lista de comandos (crear tabla, hacer login"), sino un resultado funcional verificable por el usuario final.

### 2. Distribución de Roles Operativos en el Equipo:

- **Quién Prioriza (Product Owner): Aldo Yaranga.** Define qué requerimientos aportan mayor valor a las municipalidades y empresas turísticas, establece el orden del backlog y aprueba los criterios de aceptación.
- **Quién Facilita (Scrum Master / Agile Coach): Carolay Ccama.** Vela por el cumplimiento de los límites WIP en Trello, remueve impedimentos externos y modera las sesiones de revisión.
- **Quién Construye (Tech Lead / Dev Lead): Noe Franklin Choquenaira.** Diseña la arquitectura del software, programa el núcleo de la lógica de negocio, integra la base de datos y valida las pruebas unitarias.

### 3. Dos Evidencias Objetivas para Considerar el Trabajo Hecho (Definition of Done):

- **Evidencia 1 (Demostración Funcional en Vivo):** Prueba punta a punta en el navegador donde un participante ingresa sus datos, el sistema valida que hay vacantes disponibles, confirma la inscripción y renderiza el código QR en pantalla.
- **Evidencia 2 (Garantía de Calidad de Código):** *Pull Request* en GitHub revisado y aprobado por dos miembros del equipo, sin advertencias de linters y con el script de base de datos ejecutado con éxito.

## 2. Especificación Formal de Requerimientos de Software

Bajo el estándar IEEE 830 e ISO/IEC/IEEE 29148, se formalizan los requerimientos del sistema **SIGC-CUSCO** estructurados de forma incremental.

## 2.1. Requerimientos Funcionales (RF)

| Código       | Nombre del Requisito           | Descripción Funcional                                                                                                            | Prioridad (MoSCoW) |
|--------------|--------------------------------|----------------------------------------------------------------------------------------------------------------------------------|--------------------|
| <b>RF-01</b> | Autenticación y Roles          | El sistema debe autenticar a usuarios bajo esquema RBAC (Admin, Coordinador, Docente, Alumno).                                   | Must Have          |
| <b>RF-02</b> | Gestión Multientidad           | Permitir el registro y configuración independiente de municipalidades y empresas de turismo con su respectivo logo.              | Must Have          |
| <b>RF-03</b> | Parametrización de Cursos      | Crear capacitaciones definiendo título, horas pedagógicas, cupo máximo, fechas de sesión y docente asignado.                     | Must Have          |
| <b>RF-04</b> | Inscripción con DNI            | Formulario público de registro que valida el formato y unicidad del DNI peruano en tiempo real y asigna vacante.                 | Must Have          |
| <b>RF-05</b> | Asistencia Digital QR          | El docente debe registrar asistencia escaneando el código QR del alumno o mediante ingreso rápido de PIN de sesión.              | Must Have          |
| <b>RF-06</b> | Calificación y Actas           | Registrar notas en escala vigesimal (0 a 20) y calcular automáticamente la condición de aprobación final.                        | Should Have        |
| <b>RF-07</b> | Emisión Masiva de Certificados | Generar automáticamente los certificados en PDF firmados digitalmente para los participantes aprobados.                          | Must Have          |
| <b>RF-08</b> | Verificación Pública QR        | Portal web de libre acceso donde cualquier persona puede ingresar el código único o escanear el QR y ver la validez del diploma. | Must Have          |
| <b>RF-09</b> | Modo Offline de Asistencia     | La aplicación del docente debe permitir tomar asistencia sin conexión a internet y sincronizar al recuperar red.                 | Should Have        |
| <b>RF-10</b> | Reportes para Fiscalización    | Exportar padrones oficiales de participantes, asistencia y notas en formato compatible con Excel/OSCE.                           | Could Have         |

Tabla 1: Matriz de Requerimientos Funcionales priorizados bajo técnica MoSCoW.

## 2.2. Requerimientos No Funcionales (RNF) según ISO/IEC 25010

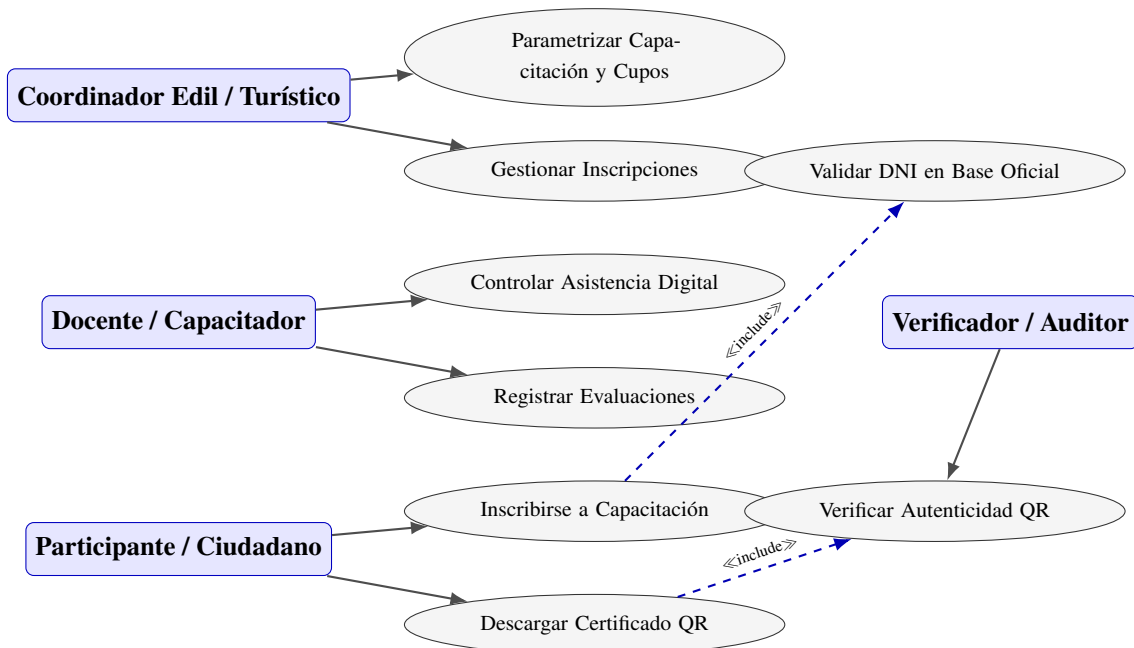
| Código | Dimensión de Calidad   | Especificación Técnica y Criterio de Aceptación                                                                                                   |
|--------|------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------|
| RNF-01 | Seguridad e Integridad | Cada certificado debe poseer un hash criptográfico SHA-256 inmutable registrado en base de datos. Contraseñas protegidas con <i>bcrypt</i> .      |
| RNF-02 | Rendimiento            | La emisión masiva de 100 certificados PDF debe completarse en segundo plano en menos de 15 segundos sin bloquear la navegación del usuario.       |
| RNF-03 | Disponibilidad         | La plataforma debe operar bajo arquitectura en la nube con un tiempo de actividad ( <i>uptime</i> ) mínimo del 99.5 % durante horarios laborales. |
| RNF-04 | Usabilidad Móvil       | El diseño debe ser 100 % adaptable ( <i>Responsive Design</i> ), permitiendo inscribirse en no más de 3 pantallas en teléfonos inteligentes.      |
| RNF-05 | Tolerancia a Fallos    | El módulo de asistencia debe utilizar almacenamiento local del navegador ( <i>IndexedDB</i> ) para tolerar cortes de red en locales comunales.    |

Tabla 2: Requerimientos No Funcionales bajo estándar de calidad ISO/IEC 25010.

## 3. Modelado Visual del Sistema (UML)

### 3.1. Diagrama de Casos de Uso del Sistema (UML)

El diagrama identifica a los cuatro actores principales y las relaciones de inclusión (*include*) que garantizan la integridad del flujo formativo.



### 3.2. Diagrama de Clases del Dominio y Multiplicidades

El modelo de clases del dominio estructura las relaciones del negocio respetando las cardinalidades del proceso formativo (ver Figura 1).

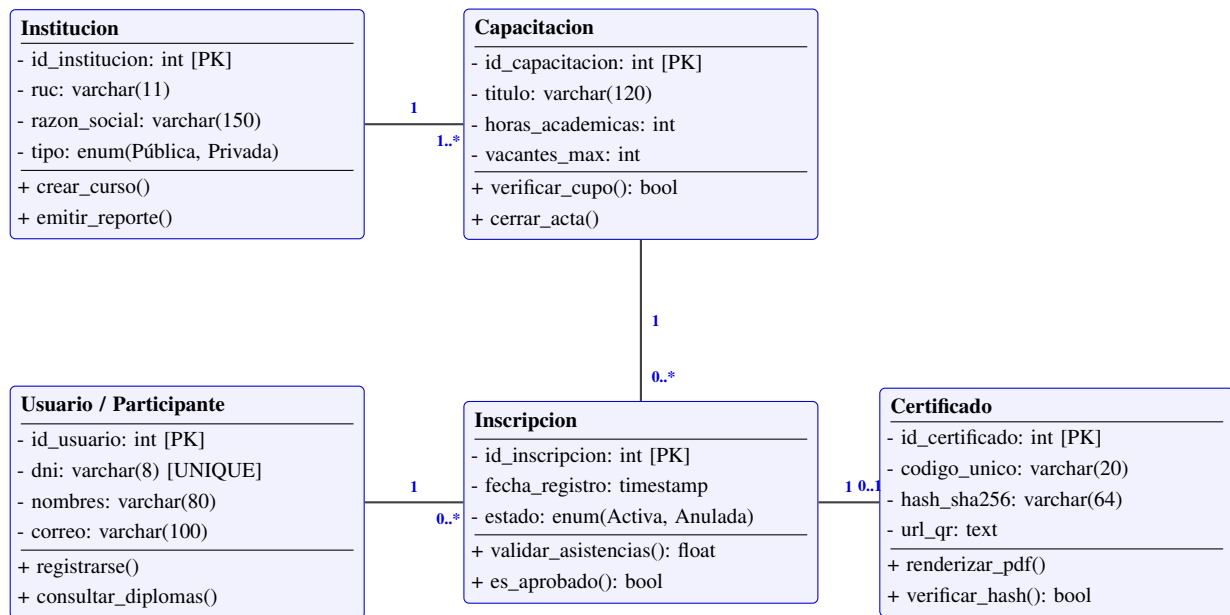


Figura 1: Diagrama de Clases del Dominio con atributos y multiplicidades formales.

## 4. Diseño de Base de Datos y Esquema Relacional

Para implementar el sistema con máxima integridad referencial y soporte transaccional ACID, se emplea el gestor relacional **PostgreSQL**.

### 4.1. Explicación Didáctica de las Conexiones Relacionales

Para explicar a nuestros compañeros cómo interactúan las tablas en la base de datos, se establecen tres patrones relacionales:

#### 1. Relación Uno a Muchos (1 a N):

- **Ejemplo:** Institución (1)  $\longrightarrow$  Capacitación (N).
- **Explicación simple:** Una sola municipalidad puede dictar muchos cursos durante el año, pero cada curso pertenece exclusivamente a una sola institución responsable. La llave primaria `id_institucion` viaja a la tabla `capacitacion` como llave foránea (*Foreign Key*).

#### 2. Relación Muchos a Muchos (N a M) resuelta por Tabla Intermedia:

- **Ejemplo:** Participante (N)  $\longleftrightarrow$  Capacitación (M).
- **Explicación simple:** Un participante puede inscribirse a varios cursos a lo largo del tiempo, y a su vez un curso tiene decenas de participantes. En bases de datos relacionales no se conectan directamente; se crea la tabla intermedia `inscripcion`, que guarda la combinación de ambos con su fecha y estado.

### 3. Relación Uno a Uno (1 a 1 Condicional):

- **Ejemplo:** Inscripción (1) → Certificado (0..1).
- **Explicación simple:** Cada registro de inscripción que cumpla con asistencia y nota aprobatoria genera exactamente un único certificado oficial. No pueden existir dos certificados distintos para la misma inscripción.

## 4.2. Esquema Físico de Base de Datos (Script DDL en PostgreSQL)

A continuación se detalla el código SQL listo para ser ejecutado:

### Script DDL PostgreSQL – SIGC-CUSCO

```
-- 1. Tabla de Instituciones (Multitenancy)
CREATE TABLE instituciones (
    id_institucion SERIAL PRIMARY KEY,
    ruc VARCHAR(11) UNIQUE NOT NULL,
    razon_social VARCHAR(150) NOT NULL,
    tipo VARCHAR(20) CHECK (tipo IN ('Publica', 'Privada')),
    logo_url TEXT,
    activo BOOLEAN DEFAULT TRUE
);

-- 2. Tabla de Usuarios (RBAC)
CREATE TABLE usuarios (
    id_usuario SERIAL PRIMARY KEY,
    dni VARCHAR(8) UNIQUE NOT NULL,
    nombres VARCHAR(80) NOT NULL,
    apellidos VARCHAR(80) NOT NULL,
    correo VARCHAR(100) UNIQUE NOT NULL,
    password_hash VARCHAR(255) NOT NULL,
    rol VARCHAR(20) CHECK (rol IN ('Admin', 'Coordinador', 'Docente', 'Alumno'))
);

-- 3. Tabla de Capacitaciones
CREATE TABLE capacitaciones (
    id_capitacion SERIAL PRIMARY KEY,
    id_institucion INT REFERENCES instituciones(id_institucion) ON DELETE CASCADE,
    id_docente INT REFERENCES usuarios(id_usuario),
    titulo VARCHAR(150) NOT NULL,
    horas_academicas INT NOT NULL CHECK (horas_academicas > 0),
    vacantes_max INT NOT NULL CHECK (vacantes_max > 0),
    fecha_inicio DATE NOT NULL,
    estado VARCHAR(20) DEFAULT 'Planificada'
);

-- 4. Tabla Intermedia: Inscripciones
CREATE TABLE inscripciones (
    id_inscripcion SERIAL PRIMARY KEY,
```

```

    id_capacitacion INT REFERENCES capacitaciones(id_capacitacion),
    id_participante INT REFERENCES usuarios(id_usuario),
    fecha_registro TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    estado VARCHAR(20) DEFAULT 'Confirmada',
    CONSTRAINT uq_participante_curso UNIQUE (id_capacitacion, id_participante)
);

-- 5. Tabla de Asistencias (Con soporte QR y Offline)
CREATE TABLE asistencias (
    id_asistencia SERIAL PRIMARY KEY,
    id_inscripcion INT REFERENCES inscripciones(id_inscripcion) ON DELETE CASCADE,
    numero_sesion INT NOT NULL,
    fecha_hora TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    estado VARCHAR(15) CHECK (estado IN ('Presente', 'Tardanza', 'Falta')),
    metodo VARCHAR(15) DEFAULT 'QR_Digital'
);

-- 6. Tabla de Certificados Criptográficos
CREATE TABLE certificados (
    id_certificado SERIAL PRIMARY KEY,
    id_inscripcion INT UNIQUE REFERENCES inscripciones(id_inscripcion),
    codigo_unico VARCHAR(32) UNIQUE NOT NULL,
    hash_sha256 CHAR(64) NOT NULL,
    fecha_emision TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    url_pdf TEXT NOT NULL
);

```

## 5. Herramientas para Diagramado Rápido y Colaborativo

Para que cualquier integrante del equipo pueda elaborar y editar los diagramas de manera simple, se recomiendan las siguientes herramientas:

### 1. dbdiagram.io (Bases de Datos Relacionales):

- **¿Por qué usarla?** Es la herramienta más rápida del mercado para crear modelos relacionales. Se escribe en lenguaje simple DBML (*Database Markup Language*) y dibuja las tablas y las líneas de conexión automáticamente.
- **Sintaxis de ejemplo para conectar tablas:**

```
Ref: capacitaciones.id_institucion > instituciones.id_institucion
```

```
Ref: inscripciones.id_capacitacion > capacitaciones.id_capacitacion
```

### 2. Mermaid.js (Diagramas UML directo en GitHub y Markdown):

- Permite crear diagramas de secuencia, casos de uso y entidad-relación sin instalar ningún software, renderizándose automáticamente en los archivos README.md de GitHub.

### 3. Draw.io / diagrams.net (Modelado Visual Libre):

- Herramienta gráfica gratuita conectada a Google Drive o VS Code para bocetar flujos y casos de uso en minutos.

## 6. Plan de Evolución Incremental e Iterativo por Sprints

Para cumplir con la premisa de evolución continua en cada clase, el desarrollo de **SIGC-CUSCO** se estructura en incrementos progresivos:

- **Incremento 1 (Sprint Inception - Actual):** Definición de visión, mapa de interesados, Lean Canvas y políticas de flujo Kanban.
- **Incremento 2 (Sprint Requerimientos y Datos):** Especificación de casos de uso detallados, diagramas relacionales y script de base de datos PostgreSQL.
- **Incremento 3 (Sprint Core Transaccional):** Implementación de portal de inscripción con validación de DNI y módulo de asistencia QR.
- **Incremento 4 (Sprint Certificación y Seguridad):** Renderizado de certificados PDF con código QR y validador público con hash SHA-256.
- **Incremento 5 (Sprint Analítica):** Dashboard institucional para municipalidades y empresas turísticas de Cusco.